L10-v8

2026年10月25日 11:33

L9 的 Agent 跑完一个 turn 就结束；L10 让它在崩溃后能恢复、在进程重启后能接着上次继续，保证"半截压缩"不会把上下文截成空——**会话持久化**——把每一轮的对话历史落盘成 JSONL。

问题	解法
崩溃恢复：进程被杀，下次启动能接着跑	每轮结束落盘，下次 `--resume` 装回
跨进程继续：同一个任务分多次跑	历史落盘，`--resume` 装回后继续调模型
改错回滚：Edit 把文件改坏	工具执行之前快照，`restore` 恢复

把"内存里的对话历史"变成"磁盘上可恢复的事件流"。

一、会话持久化

1.1 根本矛盾：无状态的推理引擎 vs 有状态的任务

大语言模型 API 本质上是一个纯函数：`f(prompt, parameters) → completion`。每次调用都是独立的、无上下文的，模型内部不保留任何跨调用的记忆。↩

但 Agent 要完成的任务是有状态的。一个“把 `src/` 下所有 TODO 汇总成清单”的任务，需要读 50 个文件、中间修改若干次、最后验证结果。如果每轮模型调用都从零开始，它永远不知道“刚才读了哪个文件”、“刚才把 `add` 改成了什么”等。

状态必须被外部化——存储对话历史、工具结果、文件状态，在每次调用时重建上下文。这就是会话持久化要解决的核心问题。

1.2 状态的三层分类

在 Agent 场景下，“状态”不是铁板一块。按生命周期和用途，至少可以分成三层：

层	内容	生命周期	持久化策略
会话内上下文	当前任务的对话历史、工具结果	任务结束即失效	JSONL 追加日志，崩溃可恢复
项目知识	项目约定、规则、长期约束	跨会话持久	人工维护的文件（CLAUDE.md / AGENTS.md）
自动记忆	Agent 自己积累的事实	跨会话持久	自动追加的文件（MEMORY.md）

关键原则：每个新会话以全新的上下文窗口开始，不带之前会话的对话历史。

跨会话传承只能靠项目知识和自动记忆。

两个理由。

> 成本：带 10 个历史会话等于每次启动烧几万 token，其中 99% 与当前任务无关。
>
> 正确性：旧会话里有大量已过时的信息，过时的正确信息比没有信息更危险——它会让 Agent 自信地做错事。

两个容易混淆的概念：“继续” 是同一个任务跨进程继续（走 JSONL 日志）；“继承” 是新任务带上旧任务的对话（不做）。“继续”与“继承”是两件事。

1.3 持久化原则

一：追加写，而不是覆写

二：每行自包含

三：先写新的，再标记老的可以丢。

二：主流技术

2.1 三种状态管理范式

范式	方法	代表	适用场景
Session Replay	每次 API 调用注入完整对话历史	LangChain / AutoGen / ChatGPT	短会话、任务边界清晰
Vector Memory	把历史做成 embedding，按语义检索	Mem0 / 各类 RAG 系统	长期记忆、跨会话知识
RAG	查询外部知识库，把检索结果塞进 prompt	企业知识库 Agent	知识密集、事实查询

当实现走的是 Session Replay 路线——把 JSONL 里的历史装回 `turn.history`，下次调用时完整注入。向量记忆和 RAG 不在本讲范围内。

2.2 持久化存储的主流选择

技术	原理	优点	缺点
JSONL 追加日志	每行一个 JSON 事件，只追加不改写	崩溃只丢一行、可 grep、可 fork、天然可回放	查询能力弱、文件会膨胀
SQLite / 数据库	结构化表存储消息	查询强、事务支持	崩溃恢复复杂、不适合“追加”语义
WAL（Write-	先写日志再改数据，崩溃	原子性保证	实现复杂，通常用于数据

Ahead Log）	时回放日志	库内部
检查点 + 事件日志	定期快照 + 快照后的增量事件	恢复快（不用从头回放）	需要管理快照生命周期

JSONL 追加日志是 Agent 会话持久化最主流的选择。Claude Code 把全部会话状态（消息、工具结果、文件历史、标题、标签、worktree 状态）都落在本地一个 JSONL 文件里，每行一条记录，只追加不改写；恢复就是把这个文件读回来。DeepSeek Harness 也把会话持久化实现为仅追加的事件日志，JSONL 是它的默认后端。gptme 在 `conversation.jsonl` 之外额外维护一个 `events.jsonl` 追加日志，用于崩溃恢复。

2.3 检查点Agent 持久化里的两种不同含义

含义一：会话状态的快照。 定期把当前完整状态存下来，恢复时从最近快照开始，再回放之后的事件。gptme 每 50 个事件写一个“checkpoint cell”，恢复时先找最新快照，再回放尾部事件。这解决的是“日志太长，从头回放太慢”的问题。

含义二：文件系统的回滚点。 在工具执行之前，把目标文件的原始内容快照下来，如果 Agent 把文件改坏了，可以 `restore` 回到修改前的状态。Claude Code 的 `file checkpointing` 追踪 `Write` / `Edit` 工具做的文件修改，允许回滚到任意历史状态。这解决的是“Agent 改错了文件”的问题。

当前示例同时实现了这两种：`SessionStore::snapshot()` 在工具执行前快照文件内容（含义二），而 JSONL 日志本身支持“半行修复”和“顺序不变式”来保证会话状态可恢复（含义一的一种简化形式——不写周期性快照，但每轮结束落盘全部新状态）。

2.4 压缩与持久化的交叉点

长会话一定会触发上下文压缩。压缩会删除旧消息、用摘要替代。持久化必须能表达“这些旧消息被取代了，不要装回上下文”。

两种方法。隐式：压缩后直接覆写整个会话文件，旧消息物理消失。简单，但崩溃时可能丢掉整个文件。显式：不删除旧消息，而是追加一条“压缩事件”，记录“从 seq X 之前的消息被摘要取代了”。当前实现采用后者——`compaction` 事件带 `replaced_upto` 字段，`load()` 时据此截断。

显式方案的优势是可审计：能看到“压缩前有哪些消息、压缩成了什么摘要”，而不是只剩摘要。代价是文件会长期保留旧数据，需要加清理策略。

新增清单

文件	变化类型	核心新增
`session_store.hpp`	新增文件	`CheckpointEvent` / `SessionMeta` / `LoadResult` / `SessionStore` / `MemoryConfig`
`session_store.cpp`	新增文件	JSONL 追加语义、崩溃恢复、检查点、fork、三层记忆
`main.cpp`	重大修改	① `--sessions` / `--session-debug` / `--resume`；② `AF_SESSION_DIR` / `AF_SESSION_ID`；③ 检查点接线；④ 落盘；⑤ 项目知识加载；⑥ 自检

三、格式：JSONL

3.1 需求

需求	含义
追加写	崩溃时最多丢最后一行，不是丢整个文件
一行一事件	可 `tail -f`、可 `grep`——可观测性是格式自带的
一行坏了只丢一行	前提是每行自包含
可回放	重放日志就是重放会话

3.2 与 SQLite 的分工

JSONL 管事件流，SQLite 管查询（会话列表、索引、token 统计）—— **大作业功能选项**。日志层 ≠ 查询层。

3.3 日志格式

```
{"t":"meta","v":1,"id":"s-123","cwd":"...","created_at":...}
{"seq":1,"t":"user","content":"把 add 改名为 sum"}
{"seq":2,"t":"assistant","content":"...","tool_calls":[...]}
{"seq":3,"t":"tool_result","call_id":"call_1","content":"..."}
{"seq":4,"t":"compaction","replaced_upto":2,"summary":"交接摘要","level":3}
{"seq":5,"t":"checkpoint","id":"cp-123","tool":"Edit","snapshots":[...]}
```

type类型：

t	进上下文?	用途
`meta`	否	会话元信息（id、cwd、创建时间）
`user` / `assistant` / `tool_result`	是	对话历史
`compaction`	看 `replaced_upto`	截断标记 + 审计
`checkpoint`	否	文件快照（回滚用）

四、完整数据流

```
启动
  |
  ├─ AF_SESSION_DIR 非空 → SessionStore::open(dir, AF_SESSION_ID)
  |    ├─ 文件存在 → scanJsonl 拿 maxSeq, resumed=true
  |    ├─ 文件不存在 → 用给定 id 建新文件, resumed=false
  |    └─ 半行修复: 最后一个字节不是 '\n' → 补一个 '\n'
  |
  ├─ --resume → session->load() → turn.history
  |
  └─ loadProjectKnowledge() → assembler.setProjectKnowledge()

循环
  |
  ├─ hooks.onToolStart(Write/Edit)
  |    └─ session->snapshot({abs}, tool) → appendCheckpoint(cp)
  |
  └─ ... 正常循环 ...

turn 结束
  |
  └─ session->append(user 消息)
     session->append(本轮新消息)
     session->appendCompaction(每个压缩事件, sessionSeqBefore)
```

三层记忆

会话内上下文	项目知识	自动记忆
JSONL + 内存	CLAUDE.md / AGENTS.md	MEMORY.md
会话结束即失	持久，人工维护	持久，自动积累

session_store.hpp

```cpp
struct MemoryConfig {
    fs::path projectDir;
    int    memoryMaxLines = 200;            // MEMORY.md 前 200 行
    size_t memoryMaxBytes = 25 * 1024;      // 或 25KB，先到者为准
};

// 加载项目知识 + 自动记忆，作为 L1 层注入
std::string loadProjectKnowledge(const MemoryConfig& cfg);

// 追加一条自动记忆
void appendAutoMemory(const MemoryConfig& cfg, const std::string& entry);
```

session_store.cpp

```cpp
// ============================================================
//  三层记忆：项目知识（CLAUDE.md / AGENTS.md）+ 自动记忆（MEMORY.md）
// ============================================================
std::string loadProjectKnowledge(const MemoryConfig& cfg) {
    std::string out;
    for (const char* name : {"AGENTS.md", "CLAUDE.md"}) {
        std::ifstream ifs(cfg.projectDir / name);
        if (!ifs) continue;
        std::ostringstream os; os << ifs.rdbuf();
        out += "# " + std::string(name) + "\n" + os.str() + "\n";
    }
    // 自动记忆：前 N 行或 N 字节，先到者为准
    std::ifstream mifs(cfg.projectDir / "MEMORY.md");
    if (mifs) {
        std::string content, line;
        int lines = 0;
        while (std::getline(mifs, line) && lines < cfg.memoryMaxLines) {
            content += line; content += '\n';
            ++lines;
            if (content.size() >= cfg.memoryMaxBytes) break;
        }
        if (!content.empty()) out += "# 自动记忆\n" + content + "\n";
    }
    return out;
}

void appendAutoMemory(const MemoryConfig& cfg, const std::string& entry) {
    std::ofstream ofs(cfg.projectDir / "MEMORY.md", std::ios::app);
    if (ofs) ofs << "- " << entry << "\n";
}
```

自动记忆的写入策略当前为空——

- 什么算“值得记住的事实”？
- 由模型决定还是由规则决定？
- 写之前要不要去重？
- 写失败了要不要回喂给模型？